Bölüm 11: Veritabanı Programlama

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week5_Database_Programming
7 Kasım 2026

İçindekiler

Bölüm 11 · Veritabanı Programlama 3

Bölümün amacı 3

Konu anlatımı 3

Veritabanı yönetim sistemi (Slayt 1–5) 3

SQL ve SQLite (Slayt 6–7) 3

Tablolar, satırlar, sütunlar ve veri tipleri (Slayt 8–11) 3

Birincil anahtar, kimlik sütunu, NULL (Slayt 12–14) 4

Bağlanma ve tablo oluşturma (Slayt 15–22) 4

INSERT ile veri eklemek (Slayt 23–27) 4

SELECT ve fetch (Slayt 28–32) 5

WHERE, LIKE, ORDER BY ve toplama fonksiyonları (Slayt 33–42) 6

UPDATE ve DELETE (Slayt 43–52) 6

Birincil anahtarlar hakkında daha fazlası (Slayt 53–57) 7

Veritabanı hatalarını işlemek ve CRUD (Slayt 58–60) 7

İlişkisel veri ve yabancı anahtarlar (Slayt 61–73) 8

Birden çok tablodan sorgu (Slayt 74–76) 8

Sık yapılan hatalar 9

Kısa tekrar soruları 9

Cevaplar 10

Bölüm 11 · Veritabanı Programlama

Bölümün amacı

Veri büyüdükçe düz dosyalar yetersiz kalır. Bu bölüm Python'dan **SQLite** veritabanıyla çalışmayı öğretir. Sonunda şunları yapabilmelisin:

- DBMS, SQL, tablo, satır, sütun, birincil anahtar ve kimlik sütunu kavramlarını açıklamak
- `sqlite3` ile bağlanmak, tablo oluşturmak/silmek ve `INSERT` ile veri eklemek
- `SELECT`, `WHERE`, `LIKE`, `ORDER BY` ve toplama fonksiyonlarıyla sorgulamak
- `UPDATE` ve `DELETE` kullanmak; veritabanı hatalarını `try/except` ile yakalamak
- İlişkisel tasarım, yabancı anahtar ve birden çok tablodan sorgu yazmak (CRUD)

Konu anlatımı

Veritabanı yönetim sistemi (Slayt 1–5)

Klasik dosyalar az veri için yeterlidir ama veri büyüdükçe basit işlemler bile zahmetli ve verimsiz olur. **DBMS** (veritabanı yönetim sistemi) büyük veriyle düzenli ve verimli çalışmak için tasarlanmış yazılımdır. Python uygulaması veriye doğrudan değil DBMS aracılığıyla erişir: komut gönderir → DBMS çalıştırır → sonucu geri yollar.

SQL ve SQLite (Slayt 6–7)

SQL DBMS'lerle çalışmanın standart dilidir; genel amaçlı bir programlama dili değildir. SQL ifadeleri Python'dan **string olarak** gönderilir. Derste kullanılan **SQLite**, Python ile birlikte `sqlite3` modülünde gelir.

Tablolar, satırlar, sütunlar ve veri tipleri (Slayt 8–11)

Veritabanı tablolardan oluşur. **Satır** bir öğenin tüm verisi, **sütun** tek bir veri parçasıdır.

SQLite tipi	Açıklama	Python karşılığı
NULL	bilinmeyen değer	`None`
INTEGER	tam sayı	`int`
REAL	gerçek sayı	`float`
TEXT	metin	`str`
BLOB	ikili büyük nesne	herhangi bir nesne

Birincil anahtar, kimlik sütunu, NULL (Slayt 12–14)

Birincil anahtar her satırda benzersiz değer tutan ve satırı tanımlayan sütundur; **asla NULL olamaz**. **Kimlik sütunu**, DBMS'in ürettiği (genellikle otomatik artan) benzersiz değerlerdir; tabloda benzersiz bir sütun yoksa birincil anahtar yapılır. `NOT NULL` kısıtı bir sütunun boş kalmasını engeller.

Bağlanma ve tablo oluşturma (Slayt 15–22)

Tipik süreç: **bağlan** (`sqlite3.connect`) → **cursor al** → işlemleri `cur.execute(sql)` ile yap → **commit** → **kapat**. `connect` veritabanı yoksa oluşturur (`':memory:'` geçici, bellekte bir veritabanıdır).

`CREATE TABLE Ad (Sutun Tip, ...)` tablo oluşturur; SQLite'ta `INTEGER PRIMARY KEY` sütunu otomatik artan kimlik sütunu olur. `IF NOT EXISTS` / `DROP TABLE IF EXISTS` güvenli oluşturma/silme sağlar.

PYTHON

```python
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('''CREATE TABLE IF NOT EXISTS Inventory (
                    ItemID INTEGER PRIMARY KEY NOT NULL,
                    ItemName TEXT,
                    Price REAL)''')
conn.commit()
cur.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
print(cur.fetchall())
conn.close()
```

ÇIKTI

```
[('Inventory',)]
```

INSERT ile veri eklemek (Slayt 23–27)

`INSERT INTO Tablo (Sutunlar) VALUES (Degerler)`. Kimlik sütunu yazılmazsa SQLite değeri kendisi üretir; tek ifadede birden çok satır eklenebilir. Değişkenlerin değerleri `?` **yer tutucularıyla** verilir (string birleştirmeyle değil).

PYTHON

```python
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('CREATE TABLE Inventory (ItemID INTEGER PRIMARY KEY NOT NULL,
↪ ItemName TEXT, Price REAL)')
cur.execute("INSERT INTO Inventory (ItemName, Price) VALUES ('Screwdriver',
↪ 4.99), ('Hammer', 12.99)")
```

PYTHON (DEVAM)

```
item_name, price = 'Wrench', 16.99
cur.execute('INSERT INTO Inventory (ItemName, Price) VALUES (?, ?)', (
↪ item_name, price))
conn.commit()
cur.execute('SELECT * FROM Inventory')
print(cur.fetchall())
conn.close()
```

ÇIKTI

```
[(1, 'Screwdriver', 4.99), (2, 'Hammer', 12.99), (3, 'Wrench', 16.99)]
```

Ek bilgi (resmi dokümantasyon): `?` yer tutucuları, değerleri SQL metnine elle eklemenin yol açabileceği SQL enjeksiyonu saldırılarını önler. Kaynak: docs.python.org/3/library/sqlite3.html (How to use placeholders to bind values in SQL queries).

SELECT ve fetch (Slayt 28–32)

`SELECT Sutunlar FROM Tablo` (`*` tüm sütunlar). Python'da iki adımlıdır: önce `execute`, sonra sonuçları getir: `fetchall()` tüm satırları liste olarak, `fetchone()` her çağrıda bir satırı tuple olarak (kalmayınca `None`) döndürür.

PYTHON

```
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('CREATE TABLE Products (Description TEXT, RetailPrice REAL,
↪ UnitsOnHand INTEGER)')
cur.execute("""INSERT INTO Products VALUES ('Dark Chocolate Bar', 2.99, 120),
               ('Chocolate Truffles', 12.99, 40), ('Mint Bar', 3.49, 80)""")
cur.execute('SELECT Description, RetailPrice FROM Products')
for row in cur.fetchall():
    print(f'{row[0]:20} {row[1]:6.2f}')
cur.execute('SELECT Description FROM Products')
print(cur.fetchone(), cur.fetchone())
```

ÇIKTI

```
Dark Chocolate Bar     2.99
Chocolate Truffles    12.99
Mint Bar               3.49
('Dark Chocolate Bar',) ('Chocolate Truffles',)
```

WHERE, LIKE, ORDER BY ve toplama fonksiyonları (Slayt 33–42)

- `WHERE kriter` yalnızca kritere uyan satırları getirir; `>`, `<`, `=`/`==`, `!=`/`<>`, `AND`, `OR`, `NOT` kullanılabilir.
- `LIKE` alt string arar; `%` birden çok karakter yerine geçer: `LIKE '%Bar%'`.
- `ORDER BY sutun` sıralar (varsayılan **artan**; azalan için `DESC`).
- `AVG`, `SUM`, `MIN`, `MAX`, `COUNT(*)` toplama fonksiyonlarıdır.

PYTHON

```
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('CREATE TABLE Products (Description TEXT, RetailPrice REAL,
↪ UnitsOnHand INTEGER)')
cur.execute("""INSERT INTO Products VALUES ('Dark Bar', 2.99, 120), (
↪ 'Truffles', 12.99, 40),
               ('Mint Bar', 3.49, 80), ('Caramel', 9.50, 20)""")
cur.execute("SELECT Description FROM Products WHERE Description LIKE '%Bar%'
↪ ORDER BY RetailPrice")
print(cur.fetchall())
cur.execute('SELECT Description FROM Products WHERE RetailPrice > 3.00 AND
↪ UnitsOnHand < 100 ORDER BY RetailPrice DESC')
print(cur.fetchall())
cur.execute('SELECT COUNT(*), MIN(RetailPrice), MAX(RetailPrice), ROUND(AVG(
↪ RetailPrice), 2) FROM Products')
print(cur.fetchone())
```

ÇIKTI

```
[('Dark Bar',), ('Mint Bar',)]
[('Truffles',), ('Caramel',), ('Mint Bar',)]
(4, 2.99, 12.99, 7.24)
```

UPDATE ve DELETE (Slayt 43–52)

`UPDATE Tablo SET Sutun = Deger WHERE kriter` var olan satırları değiştirir; `DELETE FROM Tablo WHERE kriter` siler. Etkilenen satır sayısı `cur.rowcount`'tadır. `WHERE` **unutulursa tablodaki tüm satırlar** güncellenir/silinir.

PYTHON

```
import sqlite3
conn = sqlite3.connect(':memory:')
cur = conn.cursor()
cur.execute('CREATE TABLE Products (Description TEXT, Units INTEGER)')
cur.execute("INSERT INTO Products VALUES ('Mint Bar', 5), ('Truffles', 8), (
↪ 'Dark Bar', 2)")
cur.execute("UPDATE Products SET Units = 0 WHERE Description LIKE '%Bar%'")
print('güncellenen:', cur.rowcount)
cur.execute('DELETE FROM Products WHERE Units == 0')
```

PYTHON (DEVAM)

```
print('silinen:', cur.rowcount)
conn.commit()
cur.execute('SELECT * FROM Products')
print(cur.fetchall())
```

ÇIKTI

```
güncellenen: 2
silinen: 2
[('Truffles', 8)]
```

Birincil anahtarlar hakkında daha fazlası (Slayt 53–57)

Bir tablonun **tek** birincil anahtarı olur ama birden çok sütun birleşip **bileşik anahtar** oluşturabilir: `PRIMARY KEY(RoomNumber, Building)`. SQLite tablolarında otomatik artan bir **RowID** vardır; `INTEGER PRIMARY KEY` aslında onun takma adıdır. INTEGER olmayan birincil anahtarda değeri sen vermeli ve `NOT NULL` kullanmalısın.

Veritabanı hatalarını işlemek ve CRUD (Slayt 58–60)

`sqlite3.Error` tüm veritabanı hatalarının temel sınıfıdır. Veritabanı kodu `try/except/finally` ile yazılır; bağlantı `finally`'de kapatılır. Bir veritabanı uygulamasının dört temel işlemi **CRUD**'dur: Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE).

PYTHON

```
import sqlite3
conn = None
try:
    conn = sqlite3.connect(':memory:')
    cur = conn.cursor()
    cur.execute('CREATE TABLE T (ID INTEGER PRIMARY KEY, Ad TEXT)')
    cur.execute("INSERT INTO T VALUES (1, 'a')")
    cur.execute("INSERT INTO T VALUES (1, 'b')")
except sqlite3.Error as err:
    print('Veritabanı hatası:', type(err).__name__, '-', err)
finally:
    if conn != None:
        conn.close()
```

ÇIKTI

```
Veritabanı hatası: IntegrityError - UNIQUE constraint failed: T.ID
```

İlişkisel veri ve yabancı anahtarlar (Slayt 61–73)

Aynı verinin (ör. "Marketing, New York City") birçok satırda tekrarlanması depolamayı boşa harcar ve **tutarsızlığa** yol açar. **İlişkisel tasarımda** veri tablolara bölünür: Employees tablosu departman adı yerine `DepartmentID` tutar.

Yabancı anahtar, başka bir tablonun birincil anahtarını referans eden sütundur ve **referans bütünlüğü** sağlar. ERD'de 1 → ∞ **bire-çok** ilişkiyi gösterir (bir departmanda çok çalışan). SQLite yabancı anahtarları **varsayılan olarak zorlamaz**; `PRAGMA foreign_keys=ON` gerekir.

PYTHON

```python
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('PRAGMA foreign_keys=ON')
cur.execute('CREATE TABLE Departments(DepartmentID INTEGER PRIMARY KEY NOT
↪ NULL, DepartmentName TEXT)')
cur.execute('''CREATE TABLE Employees(EmployeeID INTEGER PRIMARY KEY NOT NULL,
↪ Name TEXT,
               DepartmentID INTEGER,
               FOREIGN KEY(DepartmentID) REFERENCES Departments(
↪ DepartmentID))''')
cur.execute("INSERT INTO Departments VALUES (1, 'Marketing')")
cur.execute("INSERT INTO Employees VALUES (1, 'Jack Smith', 1)")
try:
    cur.execute("INSERT INTO Employees VALUES (2, 'Renee', 99)")
except sqlite3.Error as err:
    print('Reddedildi:', err)
```

ÇIKTI

```
Reddedildi: FOREIGN KEY constraint failed
```

Birden çok tablodan sorgu (Slayt 74–76)

Sütunlar `Tablo.Sutun` biçiminde **tam nitelikli adlarla** yazılır ve tablolar `WHERE` ile anahtarlar üzerinden eşleştirilir.

PYTHON

```python
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('CREATE TABLE Departments(DepartmentID INTEGER PRIMARY KEY,
↪ DepartmentName TEXT)')
cur.execute('CREATE TABLE Locations(LocationID INTEGER PRIMARY KEY, City
↪ TEXT)')
cur.execute('CREATE TABLE Employees(EmployeeID INTEGER PRIMARY KEY, Name TEXT,
↪ DepartmentID INTEGER, LocationID INTEGER)')
cur.execute("INSERT INTO Departments VALUES (1, 'Marketing'), (2,
↪ 'Accounting')")
```

PYTHON (DEVAM)

```
cur.execute("INSERT INTO Locations VALUES (1, 'New York City'), (2, 'Boston')")
cur.execute("INSERT INTO Employees VALUES (1, 'Jack', 1, 1), (2, 'Sonia', 2,
↪ 2), (3, 'Renee', 1, 2)")
cur.execute('''SELECT Employees.Name, Departments.DepartmentName,
↪ Locations.City
               FROM Employees, Departments, Locations
               WHERE Employees.DepartmentID == Departments.DepartmentID AND
                     Employees.LocationID == Locations.LocationID''')
for satir in cur.fetchall():
    print(satir)
```

ÇIKTI

```
('Jack', 'Marketing', 'New York City')
('Sonia', 'Accounting', 'Boston')
('Renee', 'Marketing', 'Boston')
```

Sık yapılan hatalar

- `commit()` **yapmamak:** dosya tabanlı veritabanında değişiklikler kalıcı olmaz.
- `WHERE`**'siz** `UPDATE` **/** `DELETE`**:** tüm tablo etkilenir.
- **Var olmayan tabloyu sorgulamak →** `OperationalError`.

PYTHON

```
import sqlite3
cur = sqlite3.connect(':memory:').cursor()
cur.execute('CREATE TABLE Products (Name TEXT)')
cur.execute('SELECT * FROM Urunler')
```

ÇIKTI

```
sqlite3.OperationalError: no such table: Urunler
```

- **Birincil anahtar değerini tekrar etmek →** `IntegrityError`.
- **Değerleri string birleştirmeyle SQL'e eklemek:** `?` yer tutucusu kullan.
- **SQLite'ın yabancı anahtarı kendiliğinden denetlediğini sanmak:** `PRAGMA foreign_keys=ON` gerekir.

Kısa tekrar soruları

1. SQLite'ta `INTEGER PRIMARY KEY` sütunun özelliği nedir?
2. `fetchone()` satır kalmayınca ne döndürür?
3. `WHERE Description LIKE '%Bar'` hangi satırları seçer?
4. `ORDER BY Price` hangi yönde sıralar?
5. CRUD harfleri hangi SQL komutlarına karşılık gelir?

6 Yabancı anahtar ne işe yarar? Veri tekrarının iki sorunu nedir?

Cevaplar

1 Otomatik artan kimlik sütunu olur (RowID'nin takma adı); değer verilmezse SQLite üretir.

2 `None`

3 Açıklaması "Bar" ile **biten** satırları.

4 Artan (en küçük önce).

5 Create → INSERT, Read → SELECT, Update → UPDATE, Delete → DELETE.

6 Başka bir tablonun birincil anahtarını referans ederek tablolar arasında ilişki ve referans bütünlüğü kurar. Veri tekrarı depolamayı boşa harcar ve tutarsız/çelişkili veriye yol açar.